

Московский государственный университет
имени М. В. Ломоносова

ЧИСЛЕННЫЕ МЕТОДЫ В PYTHON

Материалы компьютерного практикума

Москва, 2026

Как известно, современные языки программирования делятся на компилируемые и интерпретируемые. Конечно, задачи можно решать и на языках достаточно низкого уровня, однако для исследовательских целей, особенно при моделировании, более удобны высокоуровневые языки и специализированные программные среды.

К таким средам относятся MATLAB (и его открытый аналог Octave), Wolfram Mathematica и другие. В последние годы MATLAB развивается скорее как узкоспециализированный пакет для инженерного и научного моделирования. Более гибким и универсальным решением стал высокоуровневый язык Python, возможно уступающий MATLAB в графическом интерфейсе, но значительно превосходящий его в расширяемости и доступности.

В этом разделе мы рассмотрим базовый функционал нескольких математических библиотек Python, используемых для работы с данными различного типа и решения прикладных задач.

Материал составлен для курса читаемого П.А. Кручининим.

Глава 1

Основные библиотеки

1.1 Основные сведения. Выбор окружения

Невозможно создать пособие, которое содержало бы полное описание всех возможностей языка программирования Python или всех инструментов, связанных с его использованием. Развитие языка происходит постоянно, обновляются как стандартные модули, так и сторонние библиотеки. Поэтому в данном разделе мы приводим лишь базовые сведения, общие рекомендации и основные ориентиры. Подробности всегда следует уточнять в официальной документации, на которую мы будем ссылаться.

1.1.1 Установка интерпретатора Python

Первый шаг — установка собственно интерпретатора Python. Рекомендуемый и наиболее надёжный способ — скачать дистрибутив с официального сайта проекта: <https://www.python.org/>

На сайте Python особое внимание стоит обратить на раздел **Documentation**. Это наиболее полное и постоянно обновляемое описание стандартных модулей, встроенных функций, синтаксиса и особенностей языка. При работе над проектами мы настоятельно рекомендуем обращаться к официальной документации в первую очередь, даже если вы уже знакомы с синтаксисом: многие детали и редкие случаи описаны только там.

После установки интерпретатора пользователь получает консольное приложение `python` (или `python3` в зависимости от системы), через которое можно запускать код.

1.1.2 Выбор среды разработки

Хотя можно писать программы и в обычном текстовом редакторе, практика показывает, что среда разработки (IDE) существенно облегчает процесс: подсветка синтаксиса, автоматическое форматирование кода, навигация, встроенная проверка ошибок, система управления виртуальными окружениями и интеграция с Git экономят значительное количество времени.

Самое популярное решение для Python — **PyCharm** от компании JetBrains (<https://www.jetbrains.com/pycharm/>).

- **PyCharm Professional Edition** — расширенная версия, предназначенная для профессиональной разработки. Эта версия платная.

- **PyCharm Community Edition** — полностью бесплатная версия, включающая весь необходимый функционал для обучения и разработки большинства проектов. Именно её мы рекомендуем для учебных целей и личных проектов.

PyCharm предоставляет удобные средства для работы с проектами, запуска скриптов, установки библиотек и взаимодействия с системами контроля версий (git), использование которой мы вам советуем.

1.1.3 Альтернативные окружения

Помимо PyCharm существует множество других инструментов, позволяющих запускать и редактировать программы на Python. Перечислим наиболее распространённые из них:

- **VS Code** — лёгкая и расширяемая среда разработки. Поддержка Python обеспечивается через официальный плагин от Microsoft. Часто используется как более универсальная альтернатива PyCharm.
- **Jupyter Notebook / JupyterLab** — интерактивная среда для выполнения кода фрагментами. Особенно полезна для научных вычислений, визуализации данных и экспериментов. Является стандартом в Data Science. Этот инструмент мы будем использовать далее.
- **Anaconda / Miniconda** — другое решение, используемое для научных вычислений, анализа данных и машинного обучения. Anaconda включает собственный менеджер пакетов `conda`, позволяющий создавать изолированные виртуальные окружения и устанавливать библиотеки, которые иногда сложно собрать стандартным `pip`.

Anaconda поставляется в двух вариантах:

- **Anaconda Distribution** — полнофункциональный дистрибутив, включающий множество научных библиотек (NumPy, SciPy, Pandas, Matplotlib), утилиту **Anaconda Navigator**, а также Jupyter Notebook. Подходит пользователям, которые хотят сразу получить готовую рабочую среду для анализа данных.
- **Miniconda** — минимальная установка `conda` без дополнительных пакетов. Предпочтительна для тех, кто хочет полностью контролировать состав окружения и экономить дисковое пространство.

Anaconda упрощает работу с зависимостями и версиями Python, а также полезна там, где требуется стабильная и воспроизводимая вычислительная среда. Подробнее см. <https://www.anaconda.com/>.

- **Google Colab** — облачная версия Jupyter Notebook, доступная через браузер. Позволяет выполнять Python-код без установки интерпретатора, предоставляет бесплатный доступ к вычислительным ресурсам и GPU. Удобна, когда требуется работать с ноутбуками на разных устройствах.
- Средства интерпретации Python также доступны в облачных сервисах (например, Kaggle Notebooks), а также в составе инструментов для обучения программированию.

Каждое из перечисленных окружений имеет свои сильные стороны, и выбор зависит от задач. В рамках данного пособия основным инструментом будет PyCharm Community Edition с использованием Jupyter Notebook / JupyterLab.

Сегодня многие среды разработки включают ИИ-помощников, способных подсказывать идеи или автоматически генерировать код. Это действительно мощные инструменты, однако важно понимать их роль. ИИ полезен только тогда, когда вы уже владеете материалом настолько хорошо, чтобы уметь проверять и исправлять полученный код.

В большинстве случаев, использование ИИ не помогает освоить предмет, а лишь создаёт иллюзию понимания и затрудняет поиск логических ошибок в решении и в его реализации. Поэтому в учебных заданиях мы настоятельно рекомендуем писать код самостоятельно. Иногда ИИ можно использовать как "умный" поиск по документации, или для генерации примеров, например красивых иллюстраций. Тем не менее не забывайте, что большие текстовые модели не обладают сознанием и не гарантируют соответствия предлагаемых ими решений документации.

1.2 Справочники и пособия по Python

Поскольку Python является одним из самых популярных языков программирования, существует множество качественных учебников, онлайн-курсов и тренировочных площадок. Вы всегда можете выбрать те ресурсы, которые лучше всего подходят вашему уровню подготовки и вашим задачам. Из открытых материалов рекомендуем начать со следующего:

- Официальный тур по Python — краткое введение и документация языка. <https://docs.python.org/3/tutorial/>
- Документация Python (стандартная библиотека, справочник по функциям). <https://docs.python.org/3/>
- Учебник от Яндекс <https://education.yandex.ru/handbook/python>
- Учебник по научным вычислениям SciPy Lectures. <https://scipy-lectures.org/>
- Coursera: Programming for Everybody (Michigan University) — один из самых популярных мировых курсов. <https://www.coursera.org/learn/python>
- Stepik: Интерактивный тренажёр по Python — хороший курс для начинающих. <https://stepik.org/course/67>
- LeetCode — задачи по алгоритмам и структурам данных. <https://leetcode.com/>
- Codewars — тренировочная платформа с задачами разных уровней сложности. <https://www.codewars.com/>
- HackerRank: Python Domain — серия задач для закрепления синтаксиса. <https://www.hackerrank.com/domains/python>
- Kaggle Learn: Python — краткий интерактивный курс с задачами для анализа данных. <https://www.kaggle.com/learn/python>

1.3 Окружения, библиотеки и организация проекта

У Python существует большое количество подключаемых библиотек: как широко используемых, так и авторских, которые создают сами пользователи. Из-за разнообразия версий и зависимостей нередко возникают проблемы совместимости. Именно поэтому важно правильно организовывать рабочую среду проекта.

1.3.1 Зачем нужны виртуальные окружения

Обычно для конкретной задачи (а тем более для учебного проекта) не требуется полный набор всех существующих библиотек. Более того, разные проекты могут нуждаться в разных версиях одних и тех же пакетов. Чтобы избежать конфликтов и поддерживать воспроизводимость решений, принято создавать отдельные окружения (наборы библиотек) для каждого проекта.

При создании нового проекта в PyCharm рекомендуется выбирать опцию создания локального окружения. Обычно оно автоматически создаётся в папке `.venv` внутри проекта. Все дополнительные библиотеки затем устанавливаются именно туда через команду `pip`.

Пример. Допустим, вы создаёте проект `my_analysis`. Его структура может выглядеть так:

```
my_analysis/  
  .venv/           % локальное окружение Python  
  requirements.txt
```

1.3.2 Создание окружения вручную (альтернативный способ)

Если проект создаётся не через PyCharm, окружение можно создать вручную:

```
1 python -m venv .venv
```

Активировать его:

- Linux / macOS:

```
1 source .venv/bin/activate
```

- Windows:

```
1 .venv\Scripts\activate
```

После активации все команды `pip install` будут относиться только к этому окружению.

1.3.3 Установка библиотек

Для работы с интерактивными ноутбуками нам понадобится Jupyter Notebook. Установим его с помощью `pip`:

```
1 pip install jupyter
```

После установки можно запускать ноутбуки командами:

```
1 jupyter notebook
```

или

```
1 jupyter lab
```

Пример. Установка часто используемых библиотек:

```
1 pip install <название_библиотеки>
```

Обратите внимание, что название библиотеки для установки иногда может отличаться от популярного.

Далее все примеры будем приводить для ноутбука, поскольку он позволяет выполнять куски кода и отслеживать результат. Любой код можно также поместить в файл .py и выполнить его с помощью интерпретатора.

1.3.4 Фиксация зависимостей проекта

Чтобы не забывать, какие библиотеки были установлены, рекомендуется сохранять список зависимостей в файл:

```
1 pip freeze > requirements.txt
```

После возможно восстановить то самое окружение командой:

```
1 pip install -r requirements.txt
```

1.3.5 Git и работа с удалёнными репозиториями

Чтобы ваш проект не потерялся и чтобы можно было отслеживать изменения, рекомендуется использовать систему контроля версий `git`. Для резервного хранения и совместной работы подойдут популярные облачные платформы:

- GitHub — <https://github.com/>
- GitLab — <https://gitlab.com/>

Git позволяет:

- фиксировать каждое изменение кода,
- возвращаться к предыдущим версиям,
- работать над проектом в команде,
- хранить копию проекта в облаке.

1.4 Основные типы данных в Python

В Python существует несколько базовых типов данных, которые используются для представления чисел, текста, логических значений и кортежей объектов. Тип может быть изменяемым или неизменяемым — это одно из его ключевых свойств.

Неизменяемые (immutable) типы — это типы данных, значения которых нельзя изменить после создания объекта. Любая операция, которая выглядит как изменение

такого значения, на самом деле создаёт новый объект. К неизменяемым типам относятся, например, числа (`int`, `float`), логические значения (`bool`), строки (`str`) и кортежи (`tuple`).

Изменяемые (`mutable`) типы допускают изменение своего содержимого без создания нового объекта. Их элементы можно добавлять, удалять или заменять. К изменяемым типам относятся, например, списки (`list`), словари (`dict`) и множества (`set`).

Числовые типы

- **int** — целые числа произвольной длины. Пример: `a = 10`
- **float** — числа с плавающей точкой (вещественные). Пример: `x = 3.14`
- **complex** — комплексные числа. Пример: `z = 2 + 3j`

Логический тип

- **bool** — логические значения: `True` и `False`. Используются в условиях и выражениях сравнения.

Строки

- **str** — строки (последовательности символов). Примеры: `s = "Hello"`, `t = 'Привет'`
Строки неизменяемы: изменение символа по индексу невозможно.

Списочные структуры

- **list** — изменяемая последовательность объектов. Пример: `nums = [1, 2, 3]`
- **tuple** — неизменяемая последовательность. Пример: `p = (10, 20)`
- **range** — диапазон целых чисел, часто используется в циклах. Пример: `range(5): 0, 1, 2, 3, 4`

Кортежи с доступом по ключу

- **dict** — словарь: отображение «ключ -> значение». Пример: `d = {"a": 1, "b": 2}`
Ключи неизменяемы, значения могут быть любыми.
- **set** — множество уникальных элементов. Пример: `s = {1, 2, 3}`
- **frozenset** — неизменяемое множество. Пример: `fs = frozenset([1, 2, 3])`

Специальный тип

- **NoneType** — специальное значение `None`, обозначающее отсутствие результата. Часто используется как значение по умолчанию.

Краткие примеры

Запросить тип переменной можно с помощью функции `type()`

```
type(10)          # int
type(3.5)         # float
type("hello")     # str
type([1, 2, 3])   # list
type({"a": 1})     # dict
type(None)        # NoneType
```

1.5 Сценарии и функции в Python

В Python программа может состоять из набора сценариев (script-файлов) и функций. Сценарий служит для последовательного выполнения команд, а функции позволяют оформлять повторяющиеся вычисления в виде переиспользуемых блоков кода.

Сценарии (script-файлы)

Сценарий — это текстовый файл с расширением `.py`, содержащий последовательность команд, которые выполняются сверху вниз при запуске файла.

Пример простого сценария Рассмотрим простейший пример, для работы со скалярными величинами используем стандартную библиотеку `math`

```
1 import math
2
3 c = 3.5
4 d = a / c * math.cos(b)
5 e = a * (c * math.sin(b) + math.cos(math.pi - b))
6 f = e / d
7 print(f)
```

Функция `print(arg1, arg2, ..., argN)` ожидаемо выводит результат в консоль, или ячейку вывода. Еще несколько общих полезных команд, которые предлагаем проверить на примере `math`:

- `dir(math)` — покажет список всех функций модуля;
- `help(print)` — выведет описание функции.

Чтобы выполнить сценарий, сохраните код в файл `my1prog.py` и выполните команду:

```
1 python my1prog.py
```

также можно выполнить эту последовательность команд внутри PyCharm или в ячейке Jupiter notebook. Результат не будет отличаться. Все переменные, созданные в таком файле, существуют только в процессе его выполнения.

Обратите внимание, что синтаксис Python включает отступы. Пробелы перед выполняемой командой почти во всех случаях приведут к неисполнению команды и ошибке работы интерпретатора.

Функции

Функция — это именованный блок кода, принимающий аргументы и возвращающий результат. Функции позволяют структурировать программу, повторно использовать код и улучшать читаемость.

Определение функции выполняется с помощью ключевого слова `def`.

Пример определения функции

```
1 import math
2 def my1prog(a: float, b: float) -> tuple[float, float]:
3     """
4     вычисляет величины e и f на основе входных параметров a и b
5     параметры:
6         a (float), b (float) -- аргументы
7     возвращает:
8         tuple (e, f) -- два вычисленных значения
9     """
10    c: float = 3.5
11    d: float = a / c * math.cos(b)
12    e: float = a * (c * math.sin(b) + math.cos(math.pi - b))
13    f: float = e / d
14    return e, f
```

Обратите внимание, что `def` начинается с начала строки, а не исполняемое описание и дальнейшие строки тела функции начинаются с отступа (табуляции).

Описание функции в тройных кавычках доступно по команде `help(my1prog)`. Явное указание типов аргументов не обязательно, но позволяет IDE показывать подсказки о требуемом типе переменной при использовании функций.

Вызов функции

```
1 e1, f1 = my1prog(a, b)
```

Функции могут возвращать любое число значений. В Python это обычно реализуется через возврат кортежа.

Завершение выполнения функции

Выполнение функции завершается, когда:

- достигается команда `return`;
- выполнение доходит до конца тела функции;
- возникает ошибка.

Пример использования return

```
1 def classify(x):  
2     if x < 0:  
3         return "negative"  
4     return "non-negative"
```

Рекомендации по организации программ

- используйте сценарии для последовательных вычислений;
- оформляйте повторяющиеся действия в функции;
- структурируйте код так, чтобы каждый файл содержал логически связанные элементы;
- снабжайте функции строками описания, а код в целом — комментариями;
- передавайте необходимые данные через аргументы, избегая глобальных переменных.

Глава 2

Вектора и матрицы

Выше были приведены основные типы, на которых всё основано, но не ограничивается. В этом разделе рассмотрим реализации операций линейной алгебры. Для этого используем библиотеку **NumPy** (<https://numpy.org/>).

Установка

```
1 pip install numpy
```

Подключение

```
1 import numpy as np
```

Обратите внимание, что при подключении библиотеки можно указать любое удобное имя для обращения к ней. Использование конструкции `as np` является не обязательным, но крайне удобным: она позволяет обращаться к функциям NumPy коротко и единообразно. NumPy работает со специальным типом данных: `numpy.array`, который позволяет задавать векторы и матрицы и выполнять над ними операции линейной алгебры.

Если опустить `as np: import numpy`, то для вызова функций необходимо будет писать полное имя модуля, сравните на примере задания вектора:

import numpy

```
1 v = numpy.array([1, 2, 3])
2 print(v)
```

import numpy as np

```
1 v = np.array([1, 2, 3])
2 print(v)
```

Пример задания матрицы

```
1 A = np.array([
2     [1, 2, 3],
3     [4, 5, 6],
4     [7, 8, 9]
5 ])
6 print(A)
```

2.1 Стандартные операции с векторами и матрицами

Поэлементное сложение массивов одинаковой размерности

```

1 u = np.array([1, 2, 3])
2 v = np.array([4, 5, 6])
3
4 w = u + v      # результат: [5 7 9]

```

Если в блокноте выполнить $u + v$ без присвоения, то в ячейке output сразу отобразится результат вычисления. Аналогично, если просто написать в новой строке w , то Jupyter Notebook выведет текущее значение переменной w . Это удобно для интерактивной работы: последняя строка ячейки автоматически отображается без использования команды `print`.

Умножение на число

```

1 k = 3
2 res = k * u    # результат: [3 6 9]

```

Поэлементное умножение

```

1 C = u * v      # результат: [4 10 18]

```

Поэлементное деление

```

1 D = u / v      # результат: [0.25, 0.4 , 0.5 ]

```

Целочисленное деление

```

1 E = u // 2     # результат: [0, 1, 1]

```

Скалярное произведение

```

1 dot = np.dot(u, v)    # результат: 1*4 + 2*5 + 3*6 = 32

```

Векторное произведение Определено не для произвольных размерностей.

```

1 cross = np.cross(u, v) # результат: [-3  6 -3]

```

Транспонирование

```

1 A_T = A.T

```

Матричное умножение

```
1 # матрица A размера 3 x 4
2 A = np.array([
3     [1, 2, 3, 4],
4     [0, 1, 0, 1],
5     [5, 6, 7, 8]
6 ])
7 # матрица B размера 4 x 3
8 B = np.array([
9     [1, 0, 2],
10    [0, 1, 1],
11    [1, 0, 0],
12    [0, 1, 3]
13 ])
14
15 # матрица C размера 3 x 3
16 C = A @ B
```

Проверьте, что $A@B \neq B@A$.

Решение системы линейных уравнений

```
1 X = np.linalg.solve(A_square, B_vector)
```

Решает уравнение $AX = B$. Требование: `A_square` должна быть квадратной и невырожденной.

Решение переопределённой системы (метод наименьших квадратов)

```
1 X, *_ = np.linalg.lstsq(A, B, rcond=None)
```

Решает задачу $\min_X \|AX - B\|$.

Возведение в степень

Поэлементное возведение в степень:

```
1 C = A ** 2
```

Матричное возведение в степень:

```
1 A_k = np.linalg.matrix_power(A_square, k)
```

2.2 Некоторые стандартные функции в NumPy

2.2.1 Элементарные математические функции

- `y = np.exp(x)` — поэлементная экспонента

$$y_{ij} = e^{x_{ij}};$$

- `y = np.sin(x)` — синус;
- `y = np.cos(x)` — косинус;
- `y = np.tan(x)` — тангенс;
- `y = np.arcsin(x)` — арксинус;
- `y = np.arccos(x)` — арккосинус;
- `y = np.arctan(x)` — арктангенс;
- `y = np.sqrt(x)` — квадратный корень;
- `y = np.abs(x)` — абсолютная величина $|x|$;
- `y = np.log(x)` — натуральный логарифм;
- `y = np.log10(x)` — десятичный логарифм;
- `y = np.sign(x)` — сигнум-функция, вычисляемая по правилу:

$$y_{ij} = \begin{cases} 1, & x_{ij} > 0, \\ 0, & x_{ij} = 0, \\ -1, & x_{ij} < 0. \end{cases}$$

Все функции корректно работают и со скалярами, и с массивами любой размерности. При передаче массива результат всегда имеет ту же размерность, что и аргумент. Для работы со скалярными величинами можно использовать библиотеку `math`.

Пример.

```
1 x = np.linspace(0, np.pi, 5)
2 y = np.sin(x)
```

2.2.2 Функции округления и им сопутствующие

- `y = np.round(x, n)` — округление элементов массива `x` до `n` знаков после запятой. Если `n` не указан, округление производится до ближайшего целого.
- `y = np.floor(x)` — округление вниз (к наибольшему целому, не превосходящему `x`).
- `y = np.ceil(x)` — округление вверх (к наименьшему целому, не меньшему `x`).
- `y = np.trunc(x)` — усечение дробной части, то есть округление к нулю. Аналог `y = np.fix(x)`,
- `y = np.remainder(x, y)` — остаток от деления `x` на `y` по правилу

$$\text{rem}(x, y) = x - y \cdot \text{fix}(x/y),$$

эквивалентная операция `x % y`,

- `y = np.gcd(x, y)` — поэлементное вычисление наибольшего общего делителя (НОД) для целочисленных массивов `x` и `y`. При передаче скаляров работает как обычная функция НОД.

Для скалярных значений также доступна встроенная функция `math.gcd(x, y)`.

Все перечисленные функции применяются к каждому элементу массива отдельно, сохраняя форму входного массива.

Примеры

```
1 x = np.array([ 2.7, -3.4, 1.51 ])
2
3 np.round(x)      # [ 3. -3.  2. ]
4 np.floor(x)      # [ 2. -4.  1. ]
5 np.ceil(x)       # [ 3. -3.  2. ]
6 np.trunc(x)      # [ 2. -3.  1. ]
7 np.fix(x)        # [ 2. -3.  1. ]
```

Замечание Функции `floor` и `ceil` округляют в направлении $-\infty$ и $+\infty$ соответственно, тогда как `fix` и `trunc` округляют в направлении нуля.

2.2.3 Функции над комплексными числами

В Python комплексные числа задаются стандартным синтаксисом с использованием суффикса `j`, обозначающего мнимую единицу:

```
1 z = 6.7 + 7.897j
```

или через комбинацию действительной и мнимой частей:

```
1 a = 6.7
2 b = 7.897
3 z = a + 1j * b
```

Комплексные числа могут быть элементами массивов NumPy. Если аргументами перечисленных ниже функций являются комплексные массивы, то все операции выполняются **поэлементно**.

Ниже приведён перечень основных функций, аналогичных MATLAB:

- `y = np.angle(z)` — аргумент комплексного числа (угол в полярных координатах),

$$y = \arg(z).$$

- `y = np.conj(z)` — комплексное сопряжение,

$$y = \bar{z}.$$

- `y = np.imag(z)` — мнимая часть комплексного числа,

$$y = \Im(z).$$

- `y = np.real(z)` — действительная часть,

$$y = \Re(z).$$

- `y = np.sign(z)` — сигнум-функция для комплексного аргумента. Она равна проекции комплексного числа на единичную окружность:

$$\text{sign}(z) = \frac{z}{|z|},$$

за исключением случая $x = 0$.

Эти функции одинаково корректно работают как со скалярами, так и с массивами любой размерности, возвращая результат той же формы.

2.2.4 Формирование векторов и матриц

Создание векторов

- Вектор можно задать с помощью обычного списка Python, как было в примерах выше:

```
1 v = np.array([1, 2, 3, 4])
```

- Для построения равномерно распределённого набора точек используется функция `np.linspace`:

```
1 y = np.linspace(xmin, xmax, N)
```

N — необязательный аргумент, если его не задавать, то по умолчанию создаётся массив из 50 элементов.

- `y = np.logspace(xmin, xmax, N)` — формирует вектор (из 50 элементов, если необязательный аргумент N не задан), расположенных так, что разность логарифмов соседних точек постоянна.

Диапазоны целых чисел Для построения векторов равномерного шага удобно использовать:

```
1 x = np.arange(start, stop, step)
```

Например:

```
1 x = np.arange(1, 10, 2)    # результат выполнения [1, 3, 5, 7, 9]
```

Формирование матриц Матрица создаётся вложенными списками, каждый вложенный список — это строка:

```
1 A = np.array([
2     [1, 2.3, 4],
3     [3.4, 2, 3]
4     ])
```

В примере выше сформирована матрица размера 2×3 .

Специальные матрицы

- `np.zeros((m, n))` — матрица $m \times n$, заполненная нулями.
- `np.ones((m, n))` — матрица, заполненная единицами.
- `np.eye(n)` — единичная матрица размера $n \times n$.
- `y = np.diag(v, k)` — формирует матрицу, на k -й диагонали которой расположены элементы вектора v . При $k = 0$ получается диагональная матрица с диагональю v . При $k > 0$ вектор размещается на наддиагонали с номером k , при $k < 0$ — на поддиагонали с номером $|k|$.
- `y = np.random.rand(N, M)` — формирует массив размера $N \times M$, элементы которого являются случайными числами, равномерно распределёнными на интервале $(0, 1)$. В модуле `np.random` есть и другие варианты для случайных величин. Ознакомьтесь с ними в документации или с помощью функции `dir(np.random)` и `help`.

2.2.5 Повороты и отражения матриц

Для отражения и поворота векторов и матриц в Python используются следующие функции библиотеки NumPy:

- `np.fliplr(A)` — переставляет столбцы матрицы A симметрично относительно вертикальной оси;
- `np.flipud(A)` — переставляет строки матрицы A симметрично относительно горизонтальной оси;
- `np.rot90(A, k)` — поворачивает матрицу A на $90^\circ \times k$ против часовой стрелки. При $k = 1$ производится поворот на 90° , при $k = 2$ — на 180° , при $k = 3$ — на 270° . При $k = 0$ матрица остаётся неизменной.

Именные матрицы Именные матрицы, типа матриц Ганкеля, Тёплица, Гильберта, Адамара и т.д. реализованы в модуле SciPy (установка: `pip install scipy`). Полный список имен submodule `scipy.linalg` можно посмотреть командой `dir`, в частности в него входят: `hankel`, `toeplitz`, `hilbert`, `hadamard`, `wilkinson`.

2.2.6 Действия над векторами и матрицами

Скалярные функции: `sum`, `prod`, `min`, `max`, `mean`, `std` Если аргументом функции является вектор (одномерный массив NumPy), то результатом применения функции является скаляр. Если аргументом является матрица (двумерный массив), то результатом является вектор, j -я компонента которого равна результату применения функции к j -му столбцу матрицы.

Основные функции:

- `y = np.sum(x)` — сумма компонентов вектора;
- `y = np.prod(x)` — произведение компонентов вектора;

- `y = np.min(x)` — минимальный компонент вектора;
- `y = np.max(x)` — максимальный компонент вектора;
- `y = np.mean(x)` — среднее арифметическое компонентом вектора;
- `y = np.std(x)` — среднеквадратичное отклонение компонентом.

Если в нашем массиве встречаются NaN, которые надо игнорировать при вычислении средних, то реализовано в функциях: `np.nansum`, `np.nanprod`, `np.nanmin`, `np.nanmax`, `np.nanmean`, `np.nanstd`.

Пример работы с матрицей

```
1 x = np.array([[1, 4, 7],
2               [2, 5, 8],
3               [3, 6, 9]])
4
5 np.sum(x)          # 45 (сумма всех элементов)
6 np.sum(x, axis=0)   # [ 6, 15, 24 ] сумма по столбцам
7 np.sum(x, axis=1)   # [12, 15, 18] сумма по строкам
```

Если параметр `axis` не указан, NumPy работает по всем элементам массива, возвращая скаляр.

Медиана

```
1 y = np.median(x)
```

Возвращает медиану компонентом вектора *x*. Под медианой понимается значение, полученное после сортировки элементов вектора по возрастанию: для нечётного числа элементов выбирается значение с номером $(n + 1)/2$, для чётного — среднее арифметическое двух средних элементов.

Если аргументом является матрица, медиана вычисляется по столбцам:

```
1 np.median(A, axis=0)
```

Аналогично, для объектов содержащих NaN, используется функция `np.nanmedian`.

Проверка конечности элементов

```
1 y = np.isfinite(x)
```

Возвращает массив той же размерности, что и *x*, содержащий значение `True`, если компонент конечен (не NaN и не $\pm\infty$), и `False` в противном случае.

Проверка NaN

```
1 y = np.isnan(x)
```

Возвращает массив из `True` и `False`: `True` соответствует компонентам равным NaN.

Проверка всех элементов

```
1 y = np.all(x)
```

Возвращает True, если все компоненты массива x отличны от нуля, и False иначе.

Проверка существования ненулевых элементов

```
1 y = np.any(x)
```

Возвращает True, если хотя бы один компонент массива x отличен от нуля.

Нахождение индексов ненулевых элементов

```
1 y = np.nonzero(x)
```

Возвращает индексы ненулевых элементов массива. Для одномерного вектора результат — одномерный массив индексов. Для матрицы возвращаются два массива: индексы строк и столбцов соответствующих элементов.

Кумулятивная сумма

```
1 y = np.cumsum(x)
```

Возвращает вектор, i -й компонент которого равен сумме первых i компонент исходного вектора x . Если аргументом является матрица, операция применяется по столбцам:

```
1 np.cumsum(A, axis=0)
```

Кумулятивное произведение

```
1 y = np.cumprod(x)
```

Возвращает вектор, i -й компонент которого равен произведению первых i элементов вектора x . Для матриц применяется по столбцам:

```
1 np.cumprod(A, axis=0)
```

Сортировка

```
1 y = np.sort(x)
```

Сортирует компоненты вектора по возрастанию. Если аргументом является матрица, сортировка производится независимо в каждом столбце:

```
1 np.sort(A, axis=0)
```

Конечно-разностное дифференцирование

```
1 y = np.diff(x)
```

Для вектора длины N результатом является вектор длины $N - 1$, где

$$y_k = x_{k+1} - x_k.$$

Для матрицы вычисления ведутся по строкам (axis=0) или столбцам (axis=1), по умолчанию axis=0:

```
1 np.diff(A, axis=0)
```

n -ая разность вычисляется как:

```
1 y = np.diff(x, n)
```

Градиент

```
1 Px, Py = np.gradient(A)
```

Функция вычисляет приближённые частные производные (конечно-разностный градиент) по направлениям строк и столбцов матрицы A .

```
1 Px, Py = np.gradient(A, Dx, Dy)
```

где Dx и Dy — расстояния между точками по осям, по умолчанию считаются равными единице.

Интеграл методом трапеций

```
1 z = np.trapezoid(y, x)
```

Численно вычисляет интеграл от функции, значения которой записаны в векторе y , по точкам сетки x , используя метод трапеций:

$$z \approx \sum_{k=1}^{N-1} \frac{y_k + y_{k+1}}{2} (x_{k+1} - x_k).$$

Если аргумент x опущен:

```
1 np.trapz(y)
```

то используется равномерная сетка с шагом 1.

2.3 Нормы векторов и матриц

2.3.1 Нормы векторов и матриц

Нормы векторов. В Python нормы векторов и матриц вычисляются с помощью функции `np.linalg.norm`. Тип нормы задаётся параметром `ord`.

Специализированная функция `linalg.vector_norm(x, /, *, axis=None, keepdims=False, o`

Нормы матриц. Пусть A — матрица. Для неё используются следующие нормы.

Специализированная функция `linalg.matrix_norm(x, /, *, keepdims=False, ord='fro')`

2.3.2 Элементарные операции над матрицами

В этом разделе приведены базовые операции над матрицами, которые широко используются при численных расчётах. Второй популярной библиотекой линейной алгебры является `scipy.linalg`. Импортировать ее можно и как `import scipy` или, например, `import scipy.linalg as LA`

```
1 import numpy as np
```

Название нормы	Формула	Параметр ord
Евклидова норма (2-норма)	$\ v\ _2 = \sqrt{\sum_{i=1}^n v_i^2}$	значение по умолчанию – не указан ord=2
1-норма	$\ v\ _1 = \sum_{i=1}^n v_i $	ord=1
Бесконечная норма	$\ v\ _\infty = \max_i v_i $	ord=np.inf
Норма с показателем $-\infty$	$\ v\ _{-\infty} = \min_i v_i $	ord=-np.inf
p -норма	$\ v\ _p = \left(\sum_{i=1}^n v_i ^p \right)^{1/p}$	ord=p

Таблица 2.1: Нормы вектора и их вычисление в NumPy

Название нормы	Формула	Параметр ord
Спектральная норма (2-норма)	$\ A\ _2 = \max_i \sigma_i(A)$	ord=2
1-норма (столбцовая)	$\ A\ _1 = \max_j \sum_i A_{ij} $	ord=1
Бесконечная норма (строчная)	$\ A\ _\infty = \max_i \sum_j A_{ij} $	ord=np.inf
Мин-норма матрицы	$\ A\ _{-\infty} = \min_i \sum_j A_{ij} $	np.linalg.norm(A, ord=-np.inf)
Фробениусова норма	$\ A\ _F = \left(\sum_{i,j} A_{ij}^2 \right)^{1/2}$	ord='fro'
Ядерная норма (trace norm)	$\ A\ _* = \sum_i \sigma_i(A)$	ord='nuc'
p -норма матрицы	$\ A\ _p = \left(\sum_{i,j} A_{ij} ^p \right)^{1/p}$	отсутствует

Таблица 2.2: Нормы матриц и их вычисление в NumPy

Размерности матрицы

```

1 N, M = A.shape      # размеры матрицы A
2 L = A.size          # общее число элементов матрицы

```

Основные численные характеристики

- Определитель матрицы

```

1 y = np.linalg.det(A)

```

Только для квадратных матриц.

- След матрицы

```

1 y = np.trace(A)

```

- Обратная матрица

```
1 y = np.linalg.inv(A)
```

- Псевдообратная (в смысле Мура–Пенроуза)

```
1 y = np.linalg.pinv(A)
```

- Квадратный корень из матрицы

```
1 y = scipy.linalg.sqrtm(A)
```

обратите внимание, что в numpy нет подобной функции.

- Матрица-экспонента

```
1 y = scipy.linalg.expm(A)
```

- Матрица-логарифм

```
1 y = scipy.linalg.logm(A)
```

- Характеристический полином

```
1 v = np.poly(A)
```

Коэффициенты расположены в порядке убывания степеней.

- Собственные значения и собственные векторы

```
1 w, V = np.linalg.eig(A)
```

Массив `w` содержит собственные значения матрицы A , а матрица V содержит собственные векторы (в столбцах).

Если требуется только вектор собственных значений:

```
1 w = np.linalg.eigvals(A)
```

- Сингулярное разложение

```
1 U, S, Vh = np.linalg.svd(A)
```

Матрица A представляется в виде:

$$A = U \cdot \Sigma \cdot V^T,$$

где U и V — ортонормированные матрицы, Σ — диагональная матрица сингулярных чисел:

$$\sigma_i(A) = \sqrt{\lambda_i(A^T A)}.$$

Для восстановления диагональной матрицы сингулярных чисел используется:

```
1 Sigma = np.diag(S)
```

- Ранг матрицы

```
1 r = np.linalg.matrix_rank(A)
```

Ранг определяется как число сингулярных чисел, превышающих заданный порог числовой точности. В вычислительных процедурах очень малые значения могут рассматриваться как нулевые, что связано с особенностями численных методов.

- Число обусловленности матрицы

```
1 mu = np.linalg.cond(A)
```

Число обусловленности характеризует чувствительность решения линейной системы $Ax = b$ к малым изменениям в данных. Для нормы 2 оно выражается формулой:

$$\text{cond}_2(A) = \frac{\sigma_{\max}(A)}{\sigma_{\min}(A)},$$

где $\sigma_{\max}$ и $\sigma_{\min}$ — наибольшее и наименьшее сингулярные числа матрицы.

- Балансировка матрицы

```
1 B, T = np.linalg.matrix_balance(A, separate=True)
```

Балансировка улучшает численную устойчивость вычислений, связанных с собственными значениями матрицы. Искомая матрица T выбирается так, чтобы элементы изменённой матрицы

$$B = T^{-1}AT$$

были сбалансированы по величине, что уменьшает влияние разницы порядков чисел на вычислительные процедуры.

- Нулевое подпространство

```
1 Z = scipy.linalg.null_space(A)
```

Функция вычисляет ортонормированный базис нулевого подпространства матрицы. Столбцы матрицы Z образуют набор векторов, удовлетворяющих условию:

$$AZ = 0.$$

Если матрица имеет рациональные элементы, найденный базис может не быть рациональным, так как вычисления выполняются в действительной арифметике.

- QR-разложение

```
1 Q, R = np.linalg.qr(A)
```

Матрица A представляется в виде:

$$A = QR,$$

где Q — ортонормированная матрица, R — верхнетреугольная.

- LU-разложение

```
1 P, L, U = scipy.linalg.lu(A)
```

Матрица представляется в виде:

$$PA = LU,$$

где P — матрица перестановок, L — нижнетреугольная матрица, U — верхнетреугольная.

Для ознакомления рассмотрим пример вычислительно плохо обусловленных матриц. Классическим примером такой матрицы является матрица Гильберта, элементы которой определяются формулой:

$$H_{ij} = \frac{1}{i + j - 1}, \quad i, j = 1, \dots, n,$$

где n — порядок квадратной матрицы, i, j — номера строк и столбцов.

Матрица Гильберта является плохо обусловленной с точки зрения обращения: даже небольшие изменения элементов приводят к существенным изменениям в инверсии и в решениях линейных систем.

В Python матрицу Гильберта можно сформировать следующим образом:

```
1 import numpy as np
2 import scipy
3
4 A = scipy.linalg.hilbert(6)
```

Рассмотрим матрицу A Гильберта размера 6×6 и матрицу B , которая совпадает с A во всех элементах, кроме одного. Пусть значение элемента $B_{1,6}$ отличается от соответствующего элемента $A_{1,6}$ на 1%. Такие малые возмущения могут привести к существенным изменениям результатов вычислений, что демонстрирует плохую обусловленность:

```
1 B = A.copy()
2 B[0, 5] *= 1.01    # изменение на 1%
```

Рассмотрим отличия при расчете характеристик плохо обусловленных матриц. Разницу между обратными матрицами выразим в долях от матрицы A

```
1 result = (np.linalg.inv(B) - np.linalg.inv(A)) / np.linalg.inv(A)
```

Собственные числа

```
1 wA, _ = np.linalg.eig(A)
2 wB, _ = np.linalg.eig(B)
3 print(wA, wB)
4 d = np.linalg.cond(A)
5 print(d)
```

В приведённом примере относительная ошибка вычисления компонент обратной матрицы составляет более 100% при ошибке задания одной из компонент матрицы в 1%. При этом собственные числа обеих матриц на первый взгляд вычисляются

вполне приемлемо. Основным признаком возможного неблагополучия в данном примере следует считать большое значение числа обусловленности и малое значение определителя матрицы.

ЭТОТ РАЗДЕЛ НАДО ПЕРЕДЕЛАТЬ: КАКОЙ ПРИМЕР МАТРИЦЫ ТУТ СДЕЛАТЬ

Рассмотрим возмущение матрицы a . Пусть матрица b совпадает с a , за исключением одного элемента, который изменён на 1%:

```
1 b = a
2 b(5,1) = b(5,1) * 1.01
```

Вычислим собственные значения матриц a и b :

```
1 [eig(a), eig(b)]
```

В результате получаем следующие значения:

-0.0408	-0.9201	+50.8135 i
-0.0119	+0.0386 i	-0.9201
-0.0119	-0.0386 i	-0.0000
0.0323	+0.0230 i	0.9201
0.0323	-0.0230 i	0.9201

Число обусловленности матрицы a равно:

```
1 cond(a)
```

$$\text{cond}(a) = 2.0253 \times 10^{18}.$$

Собственные числа матриц a и b в данном примере отличаются примерно на два порядка. Признаком такого поведения собственных чисел служит чрезвычайно большое значение числа обусловленности, порядка 10^{18} .

В заключение отметим, что в современных вычислительных пакетах реализован богатый набор функций, предназначенных для приведения матриц к различным каноническим формам, таким как формы Жордана, Фробениуса, Шура, Хессенберга, а также широко используемые LU - и QR -разложения.

Глава 3

Условные операторы, циклы, переключатели

В этом разделе рассматриваются основные элементы программирования на языке Python: логические переменные, условные операторы, циклы и конструкции ветвления, используемые при решении вычислительных и прикладных задач.

3.1 Логические переменные

В языке Python логические выражения формируются с помощью операций сравнения. Результатом таких операций является логическое значение типа `bool`, принимающее одно из двух значений: `True` (истина) или `False` (ложь).

3.1.1 Операции сравнения

В Python определены следующие операции сравнения:

Оператор	Описание
<code><</code>	меньше
<code>></code>	больше
<code><=</code>	меньше либо равно
<code>>=</code>	больше либо равно
<code>==</code>	равенство
<code>!=</code>	неравенство

Эти операции могут применяться как к скалярным величинам, так и к массивам (например, объектам NumPy). В случае массивов сравнение выполняется **поэлементно**, а результатом является массив логических значений той же размерности.

Аналогично определены логические операции в языке Python. При работе с массивами NumPy логические операции выполняются **поэлементно** и возвращают массив логических значений типа `bool` той же размерности.

- `&` — логическое И (AND). Результатом операции `A & B` является массив той же размерности, каждый элемент которого равен `True`, если соответствующие элементы массивов `A` и `B` оба истинны (не равны `False`), и `False` в противном случае.

```
1 C = A & B
```

- `|` — логическое ИЛИ (OR). Результатом операции `A | B` является массив той же размерности, каждый элемент которого равен `True`, если хотя бы один из соответствующих элементов массивов `A` или `B` истинен, и `False` в противном случае.

```
1 C = A | B
```

- `~` — логическое НЕ (NOT). Результатом операции `~A` является массив той же размерности, каждый элемент которого равен `True`, если соответствующий элемент массива `A` ложен, и `False` в противном случае.

```
1 C = ~A
```

Следует отметить, что для скалярных логических выражений в Python используются операторы `and`, `or` и `not`, тогда как операторы `&`, `|` и `~` предназначены для поэлементных логических операций над массивами.

Например,

```
1 import numpy as np
2
3 a = np.array([[1, 2],
4               [3, 4]])
5 b = np.array([[4, 2],
6               [3, 1]])
7 c = a >= b
8 c
```

```
1 Output: array([[False,  True],
2               [ True,  True]])
```

3.2 Условный оператор

В языке Python для организации условных переходов используется *условный оператор*, который в общем виде записывается следующим образом:

Запись условного оператора имеет следующий вид:

```
1 if logical_variable_1:
2     instructions_1
3 elif logical_variable_2:
4     instructions_2
5 else:
6     instructions_3
```

Отметим, что условный оператор применим только к скалярным величинам. Условия всегда отделяются от инструкций двоеточием. Инструкции, содержащие одну строку, могут писаться в одной строке с условием. В общем случае инструкции начинаются с отступа. В качестве логической переменной может использоваться также

логическое выражение, то есть правая часть присваивания, описанного в предыдущем пункте.

Под инструкциями здесь понимается набор команд, которые будут выполняться, если все элементы матричной логической переменной ненулевые.

В приведённом условном операторе конструкции, начинающиеся с `elif` и `else`, являются необязательными.

Проиллюстрируем использование условий на двух примеров:

Пример 1

```
1 a = np.eye(2)
2 b = np.ones((2,2))
3 d = np.array([[1, 1],
4               [1, 0]])
5 if 'd' in locals():
6     c=a
7 else:
8     c=b
9 print(c)
```

Обратите внимание, что `d` — не логическая переменная. Функция `locals()` возвращает словарь, содержащий переменные заданные в текущей сессии. Описание вы можете запросить через `help(locals)`.

Пример 2

```
1 if (a <= b).all():
2     c=a
3 else:
4     c=b
5 print(c)
```

Результат сравнения `a <= b` — матрица и оператор условия к ним не применимы. Поэтому уточним наше условие: `(a <= b).any()` вернет `True`, если хотя бы для одного элемента условие справедливо, `(a <= b).all()` — если условие верно для всех элементов матриц.

Само собой, всё распространяется обычным образом на случай скалярных величин, и именно такое традиционное обращение оператора сравнения можно встретить в программах:

```
1 if (a < b).all():
2     c = a
3 elif (a == b).all():
4     c = 1
5 else:
6     c = b
```

3.3 Организация циклов

При программировании в пакетах используются циклы двух видов:

- циклы типа `for`;
- циклы типа `while`.

Первая из этих конструкций — цикл типа `for` — используется для организации вычислений с заданным числом повторяющихся циклов. Конструкция такого цикла имеет вид:

```
1 for var in expression:
2     instructions
```

В качестве `expression` в предлагаемой конструкции можно использовать произвольную матрицу размера $m \times n$. В этом случае `instructions` тела цикла будут повторяться n раз, причем параметр `var` принимает последовательно векторные значения, совпадающие со столбцами матрицы. Это хорошо видно из следующего примера.

```
1 a = np.array([[1, 2, 3],
2               [4, 5, 6]])
3 for k in a.T:
4     print(k)
```

При написании кода иногда надо предусмотреть условия или цикл, но его тело пока невозможно выполнить. В таком случае вместо тела цикла можно поставить `pass`.

Однако чаще всего “выражение” записывается в более традиционном виде, и обращение к циклу `for` в программах имеет вид:

```
1 a = np.zeros((3, 3), dtype=int)
2
3 for p in range(1, 4):
4     for q in range(1, 4):
5         a[p-1, q-1] = p + q
```

В результате выполнения script-файла с указанным набором операторов будет получена матрица

$$\begin{pmatrix} 2 & 3 & 4 \\ 3 & 4 & 5 \\ 4 & 5 & 6 \end{pmatrix}.$$

Второй тип циклов `while` позволяет выполнять инструкции в теле цикла до тех пор, пока выполняется заданное условие. Конструкция такого цикла имеет вид:

```
1 while logical_variable:
2     instructions
```

В этом выражении `logical_variable` имеет тот же смысл, что и для оператора `if` в 3.2, и вычисления производятся до тех пор, пока в качестве логической переменной используется ненулевое значение.

Так же как и в конструкции `for`, в конструкции `while` обязательно используется условный оператор. В связи с этим приведём традиционный и нетрадиционный примеры использования цикла `while`.

Традиционный пример:

```

1 p = 1
2 k = 0
3 while p < 10:
4     p = p * 2
5     k = k + 1
6 print(f'k = {k}, p = {p}')
```

вычисляется минимальное значение показателя степени k такое, что $2^k \geq 10$. Очевидно, что по окончании работы цикла $k = 4$ и $p = 16$.

Менее традиционным следует считать пример вида

```

1 a = np.array([2, 5, 3, 4, 5])
2 while a.any():
3     a = a - 1
```

В этом примере заданная матрица a преобразуется следующим образом: все элементы уменьшаются на каждом шаге на 1 до тех пор, пока один из них не станет равным нулю. На примере вывода продемонстрирован вывод переменных внутри текстовой строки. Таким образом, процедура завершается, если хотя бы одна из компонент матрицы a — положительное целое число — обратится в 0. Если это не так, то произойдёт заикливание программы.

Для защиты от такой ситуации можно использовать оператор **break**, который прекращает действие последнего по вложению цикла **for** или **while**, внутри которого расположен оператор **break**. Предыдущий пример, прерванный таким оператором, переписанный в файле, примет вид:

```

1 a = np.array([2, 5, 3, 4, 5])
2 while a.any():
3     a = a - 1
4     if (a < 0).any():
5         break
```

3.4 Переключатель

Конструкция переключателя используется для множественного выбора (или ветвления). В языке Python аналогичная конструкция реализуется с помощью условного оператора **if-elif-else** и имеет вид:

```

1 if switch_expression == case_expression:
2     instructions_1
3 elif switch_expression in {case_expression_1,
4                             case_expression_2,
5                             case_expression_3}:
6     instructions_2
7 else:
8     instructions_3
```

В качестве **switch_expression** может использоваться любой из описанных ранее объектов Python. Если текущее значение этого выражения совпадает с одним из **case_expression**, выполняются инструкции соответствующего блока. Если ни одно из условий не выполняется, выполняются инструкции блока **else**.

Если выражение не подошло, выполняются инструкции, следующие за оператором `else`.

В качестве примера рассмотрим сценарий `asw.py`:

```
1 var = None
2
3 if var in {1, 2, 12, 'January', 'February', 'December'}:
4     print('Winter')
5 elif var in {3, 4, 5, 'March', 'April', 'May'}:
6     print('Spring')
7 elif var in {6, 7, 8, 'June', 'July', 'August'}:
8     print('Summer')
9 elif var in {9, 10, 11, 'September', 'October', 'November'}:
10    print('Autumn')
11 else:
12    print('It is not a month.')
```

Обращение к этому сценарию приводит к следующим результатам:

```
1 var = 3
2 Spring
3
4 var = 'May'
5 Spring
6
7 var = 10
8 Autumn
9
10 var = 'Autumn'
11 It is not a month.
```

Глава 4

Графика

4.1 Двумерная графика

4.1.1 Графики в декартовых координатах

Для построения графиков в Python существует несколько стабильных библиотек, позволяющих строить разнообразные, красивые графики для всех возможных запросов. Мы рассмотрим несколько наиболее применимых в практике математического моделирования библиотек:

- **matplotlib** — <https://matplotlib.org/>
устанавливается командой

```
1 pip install matplotlib
2
```

- **plotly** — <https://plotly.com/python/>
устанавливается командой

```
1 pip install plotly
2
```

Отметим, что библиотека **matplotlib** ориентирована в первую очередь на статические графики, тогда как **plotly** предоставляет расширенные возможности интерактивного анализа данных.

Для представления результатов статистических исследований рекомендуем обратить внимание на библиотеку **Seaborn**.

В Python при отображении графиков возникает объект нового типа — графическое окно, который имеет ряд свойств, может передаваться в функцию или возвращаться из нее.

matplotlib

Основная функция библиотеки **matplotlib** — **matplotlib.pyplot.plot**. Она требует одного обязательного аргумента — значений по оси ординат, и множество необязательных аргументов, позволяющих настроить отображение графика.

Загрузка библиотек, которые понадобятся в примерах этого раздела

```
1 import numpy as np
2 import matplotlib.pyplot as plt
```

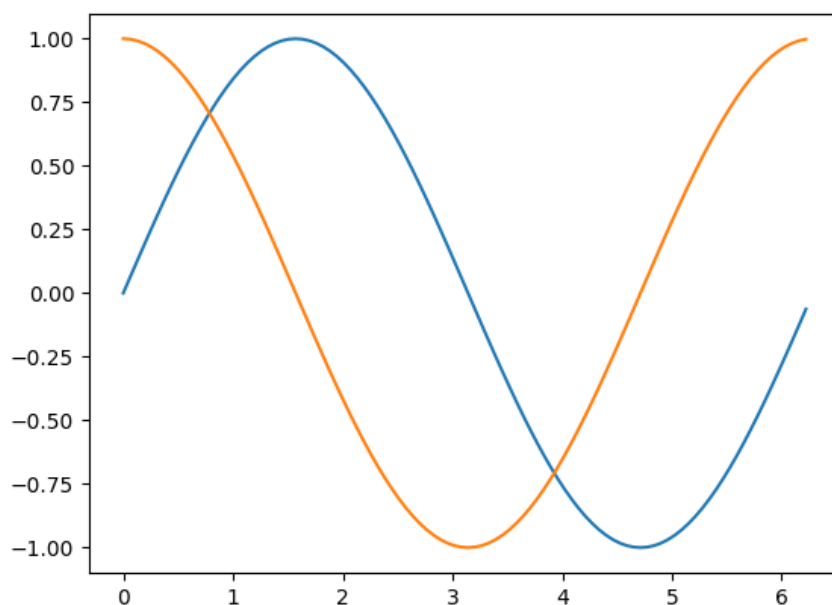
- `plt.plot(x, y)` строит график, откладывая по оси абсцисс значения компонент массива `x` и по оси ординат значения компонент массива `y` той же длины. По умолчанию, график строится как ломаная, проходящая через заданные точки.
- `plt.plot(y)` строит график, откладывая по оси ординат значения компонент массива `y`, а по оси абсцисс — номера соответствующих компонент.

```
1 t = np.arange(0.0, 2*np.pi, 2*np.pi/100)
2 plt.plot(t, np.sin(t))
3 plt.plot(t, np.cos(t))
4 plt.show()
```

Чтобы вставить результат в этот текст мы воспользуемся функцией сохранения рисунка

```
1 plt.savefig("trig_functions.png", format="png")
```

Эта команда сохраняет графическое окно в текущем состоянии. Применяется строго до `plt.show()`



Альтернативное построение того же графика, но с формированием массива.

```
1 Y = np.column_stack([np.sin(t), np.cos(t)])
2 plt.plot(t, Y)
3 plt.show()
```

А примерах мы явно не указывали графическое окно. Приведем тот же пример явно задав имя графического окна:

```

1 t = np.arange(0.0, 2*np.pi, 2*np.pi/100)
2
3 fig, ax = plt.subplots()          # явное создание объекта Figure и Axes
4 ax.plot(t, np.sin(t), label='sin(t)')
5 ax.plot(t, np.cos(t), label='cos(t)')
6 ax.legend()
7 plt.show()

```

Применимые команды Axes показаны в примере, а вот сохранение рисунка будет применяться к переменной `fig`: `fig.savefig(...)`

Команда `plt.show()` предназначена для отображения построенного графика на экране и завершения этапа его формирования.

При вызове этой команды открывается графическое окно, в котором отображаются все элементы, добавленные к графику ранее (линии, подписи осей, легенда, сетка и т. д.). Кроме того, запускается цикл обработки событий графического окна, что позволяет пользователю масштабировать изображение, перемещать его и закрывать окно. Этот функционал зависит от используемого окружения.

В случае выполнения программы в виде сценария (py-файла) использование команды `plt.show()` является обязательным, так как без неё графическое окно не будет отображено. В интерактивных средах (например, Jupyter Notebook) график может отображаться автоматически, однако явный вызов `plt.show()` рекомендуется для повышения наглядности и переносимости кода.

После выполнения команды `plt.show()` построение текущего графика считается завершённым.

plotly

Пакет `plotly` удобен для блокнотов и исследовательских целей, но стоит отметить его более высокие требования к ресурсам. Реализуем пример выше средствами `plotly`:

```

1 import numpy as np
2 import plotly.graph_objects as go
3 import plotly.io as pio
4
5 pio.renderers.default = "notebook" # для отображения в Jupyter lab
6
7 t = np.arange(0.0, 2*np.pi, 2*np.pi/100)
8
9 fig = go.Figure()
10 fig.add_trace(go.Scatter(x=t, y=np.sin(t), mode='lines', name='sin(t)'))
11 fig.add_trace(go.Scatter(x=t, y=np.cos(t), mode='lines', name='cos(t)'))
12 fig.show()

```

Получаемый в данном случае объект `fig` динамический, имеет большой размер, сохраняется командой `fig.write_image(...)`, а также с сохранением функционала зума в html: `fig.write_html("file_name.html")` .

- `go.Scatter(x=x, y=y, mode='lines')` строит график по точкам (`x_i`, `y_i`) и соединяет их линиями.

- При построении нескольких графиков на одной оси добавляются несколько трасс (несколько вызовов `fig.add_trace(...)`).

4.1.2 Дополнительные аргументы и настройки

С дополнительными аргументами мы рекомендуем ознакомиться в актуальной документации. В примере ниже мы постарались собрать весь регулярно используемый функционал. Реализация с помощью `matplotlib.pyplot`:

```

1 plt.figure(figsize=(6, 4)) # явное задание графического окна
2                             # с заданием размеров
3 plt.plot(
4     t, y,
5     color='red',             # цвет линии
6     linestyle='--',         # тип линии
7     linewidth=2.0,          # толщина линии
8     marker='o',             # маркеры
9     markersize=6,           # размер маркеров
10    markerfacecolor='white', # цвет заливки маркера
11    markeredgecolor='black', # цвет контура маркера
12    alpha=0.8,               # прозрачность
13    label='sin(t)',          # подпись для легенды
14 )
15
16 plt.title('Example: matplotlib plt.plot') # названия
17 plt.xlabel('t')                       # подпись горизонтальной оси
18 plt.ylabel('y')                       # подпись вертикальной оси
19 plt.grid(True)                        # включение сетки
20 plt.legend()                          # отображение подписей
21 plt.tight_layout()                   # автоматические размеры и отступы
22 plt.show()

```

Аналогичный рисунок с использованием библиотеки `plotly`

```

1 t = np.linspace(0, 2*np.pi, 100)
2 y = np.sin(t)
3
4 fig = go.Figure()
5 fig.add_trace(go.Scatter(
6     x=t,
7     y=y,
8     mode='lines+markers', # линии + маркеры
9     name='sin(t)',        # подпись в легенде
10    opacity=0.8,           # прозрачность
11    line=dict(
12        color='red',       # цвет линии
13        width=2,           # толщина линии
14        dash='dash'        # тип линии: solid, dash, dot, dashdot
15    ),
16    marker=dict(
17        symbol='circle',   # вид маркера
18        size=6,            # размер маркера

```

```

19     color='white',      # цвет маркера
20     line=dict(
21         color='black',  # цвет контура маркера
22         width=1
23     )
24 )
25 ))
26
27 fig.update_layout(
28     title='Example: plotly go.Scatter',
29     xaxis_title='t',
30     yaxis_title='y',
31     showlegend=True
32 )
33
34 fig.show()

```

Таблица 4.1: Обозначения цветов, маркеров и типов линий в `matplotlib`

Категория	Обозначение	Описание
Цвет линии	'y'	жёлтый
	'm'	фиолетовый (magenta)
	'c'	голубой (cyan)
	'r'	красный
	'g'	зелёный
	'b'	синий
	'w'	белый
	'k'	чёрный
Маркер	'o'	окружность
	'x'	крест
	'+'	плюс
	'*'	звёздочка
	's'	квадрат
	'd'	ромб
	'v'	треугольник вниз
	'^'	треугольник вверх
	'<'	треугольник влево
	'>'	треугольник вправо
	'p'	пятиугольник
	'h'	шестиугольник
Тип линии	'_'	сплошная
	':'	пунктир
	'-.'	штрих-пунктир
	'--'	штриховая

Отметим, что отображаемые в одном окне графики могут иметь разную длину и строится поэтапно, в том числе внутри циклов:

```

1 t = np.linspace(0, 2*np.pi, 100)
2 plt.plot(t, np.sin(t), 'b-')

```

```

3
4 t = np.linspace(0, np.pi, 100)
5 plt.plot(t, np.cos(t), 'r-')
6 plt.show()

```

Для `plotly` изменяется только синтаксис. Мы не рекомендуем смешивать процесс моделирования и отрисовки в одних и тех же функциях.

При необходимости очистить окно, например, если результат уже сохранен, и в этом же окне планируется отрисовка результата следующего этапа, то вам пригодится команда `plt.clf()`.

Для получения информации о текущем числе графиков в окне можно использовать `len(plt.gca().lines)`, которая возвращает число типа `int`.

Как уже обсуждалось раньше, заключительная команда `plt.show()` завершает построение графика и отображает его на экране. По умолчанию при каждом новом запуске графического окна предыдущее изображение очищается.

4.1.3 Недекартовы оси

В ряде задач, в частности в теории управления и акустике, часто используются логарифмические оси координат, когда по одной или нескольким осям откладываются значения не самих величин, а их логарифмов.

Построение таких графиков в `matplotlib` осуществляется функциями:

- `plt.semilogx(x, y)` — логарифмический масштаб по оси абсцисс;
- `plt.semilogy(x, y)` — логарифмический масштаб по оси ординат;
- `plt.loglog(x, y)` — логарифмический масштаб по обеим осям.

Все эти функции допускают те же формы обращения, что и ранее рассмотренная функция `plt.plot`.

Для построения графиков в полярных координатах, задаваемых углом φ и радиусом ρ , в `matplotlib` используется функция

```

1 plt.polar(phi, rho)

```

В качестве примера приведем команды, позволяющие построить кривую, называемую улиткой Паскаля. Ее уравнение в полярных координатах имеет вид

$$\rho = a \cos \varphi + \ell.$$

Пусть $a = 0.5$ и $\ell = 0.15$ и требуется построить график при всех значениях φ . Пример:

```

1 phi = np.arange(0, 2*np.pi, 0.05)
2 rho = 0.5*np.cos(phi) + 0.15
3
4 plt.polar(phi, rho)
5 plt.show()

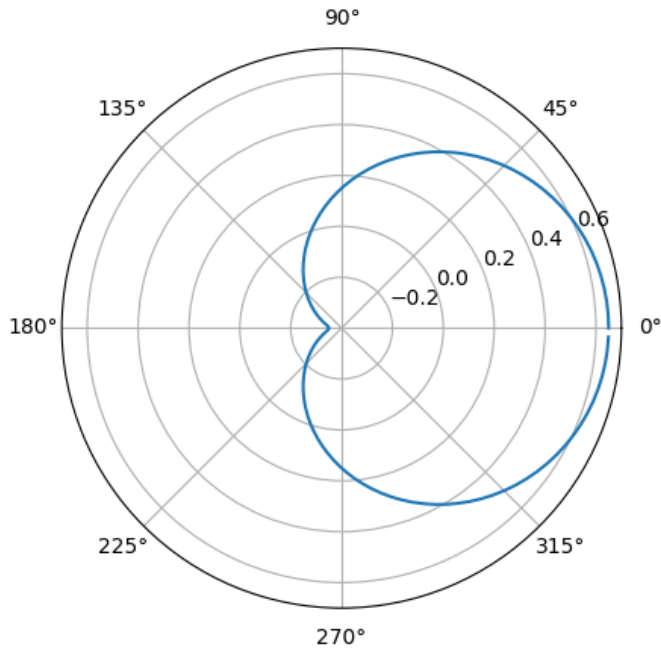
```

или с явным заданием графического объекта и сохранением изображения:

```

1 fig, ax = plt.subplots(subplot_kw={'projection': 'polar'})
2 ax.plot(phi, rho)
3 plt.show()
4 fig.savefig("polar.png", format="png")

```



Как вы могли заметить, что по умолчанию графический объект `matplotlib.pyplot.plot` не содержит координатной сетки, а `plotly` содержит. Аргумент `grid` позволяет включать сетку и настраивать ее параметры.

4.1.4 Группы рисунков

Задача отображения нескольких координатных осей — часто возникает и в ходе исследований, и при представлении результатов. На примере этой задачи разница между функционалом `matplotlib` и `plotly` будет хорошо видна.

`matplotlib`

Сначала рассмотрим построение группы статичных рисунков с помощью `matplotlib`. Один объект `Figure` может содержать несколько объектов `Axes`. Добавление и настройка отдельных областей рисования выполняется методом `fig.add_subplot(...)` (аналогично можно использовать `plt.subplot(...)`).

Продemonстрируем соответствующие возможности на следующем примере:

```

1 t = np.arange(0, 4*np.pi, 0.05)
2
3 fig = plt.figure()
4 # --- 1. декартова траектория ---
5 ax1 = fig.add_subplot(2, 2, 1)
6 ax1.plot(np.cos(t) * np.exp(-0.1*t),
7          np.sin(t) * np.exp(-0.1*t))
8 ax1.set_xlabel('distance, m')
9 ax1.set_ylabel('velocity, m/s')
10 ax1.set_title('Cartesian trajectory')
11
12 # --- 2. полярная траектория ---

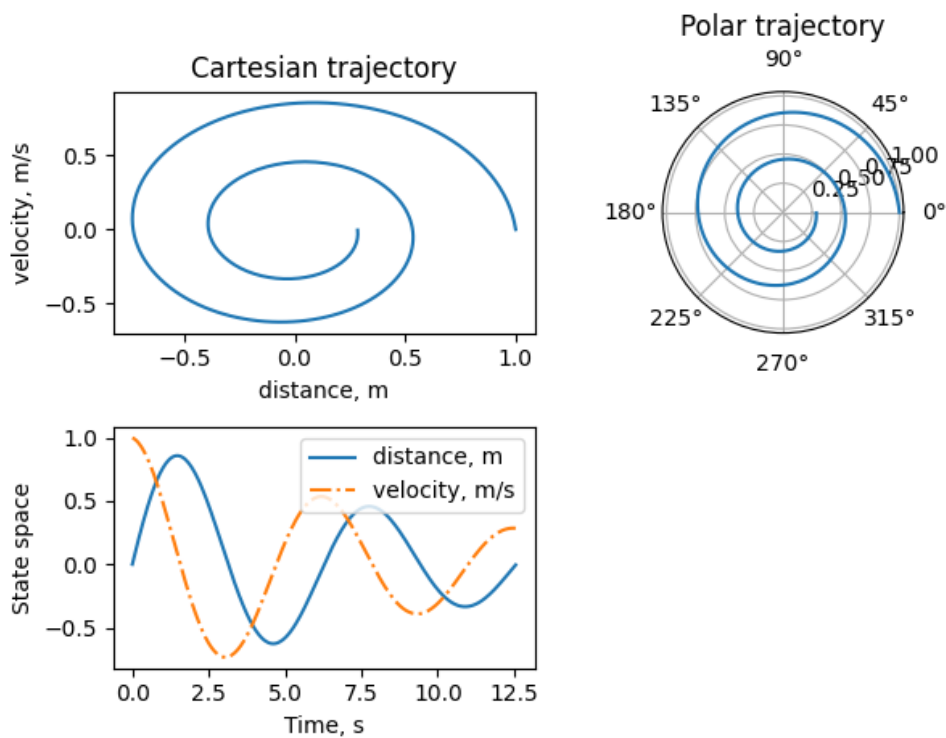
```

```

13 ax2 = fig.add_subplot(2, 2, 2, projection='polar')
14 ax2.plot(t, np.exp(-0.1*t))
15 ax2.set_title('Polar trajectory')
16 # --- 3. временные зависимости ---
17 ax3 = fig.add_subplot(2, 2, 3)
18 ax3.plot(t, np.sin(t)*np.exp(-0.1*t), '-.',
19          t, np.cos(t)*np.exp(-0.1*t), '-. ')
20 ax3.set_xlabel('Time, s')
21 ax3.set_ylabel('State space')
22 ax3.legend(['distance, m', 'velocity, m/s'])
23
24 plt.tight_layout()
25 plt.show()

```

В данном примере в одном графическом окне создаётся сетка 2×2 , а затем последовательно активируются и заполняются три области рисования: фазовый портрет, график в полярных координатах и зависимости от времени. Подписи осей и заголовки задаются методами соответствующих объектов **Axes**. Команда `plt.tight_layout()` автоматически подбирает отступы, чтобы элементы оформления (подписи, заголовки, легенда) не перекрывались.



Приведенные решения не единственные рабочие варианты. Расширяя применяемый инструментарий можем добиться более гармоничной визуализации

```

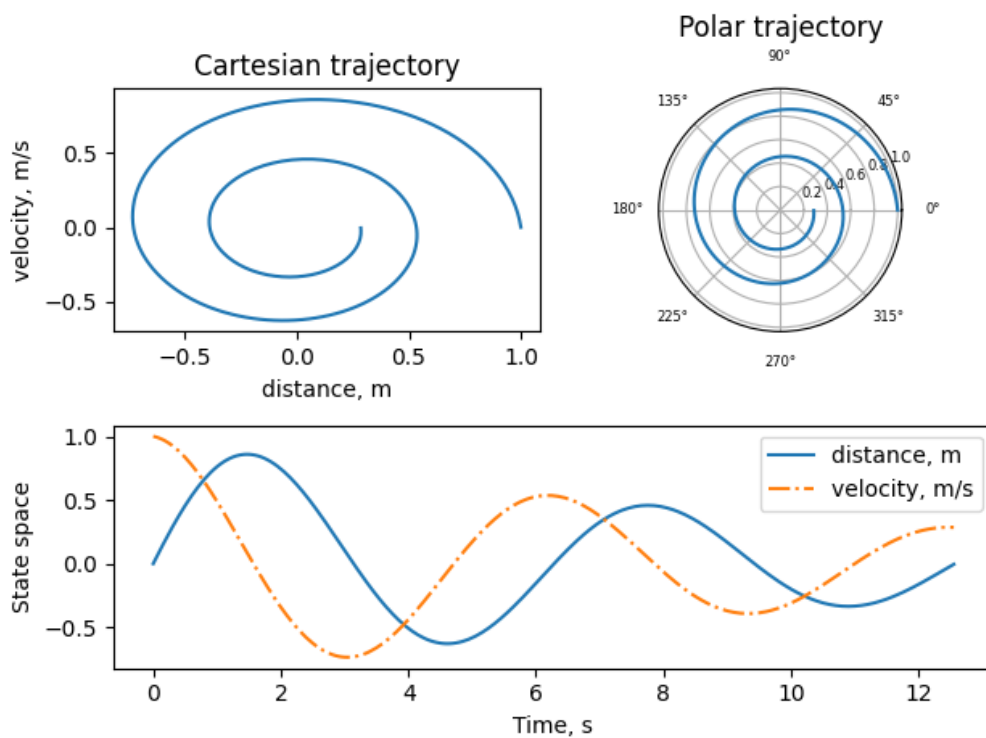
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from matplotlib.gridspec import GridSpec
4
5 t = np.arange(0, 4*np.pi, 0.05)
6

```

```

7 fig = plt.figure()
8
9 gs = GridSpec(2, 2, figure=fig)
10
11 # --- 1. декартова траектория (верхний левый) ---
12 ax1 = fig.add_subplot(gs[0, 0])
13 ax1.plot(np.cos(t) * np.exp(-0.1*t),
14         np.sin(t) * np.exp(-0.1*t))
15 ax1.set_xlabel('distance, m')
16 ax1.set_ylabel('velocity, m/s')
17 ax1.set_title('Cartesian trajectory')
18
19 # --- 2. полярная траектория (верхний правый) ---
20 ax2 = fig.add_subplot(gs[0, 1], projection='polar')
21 ax2.plot(t, np.exp(-0.1*t))
22 ax2.set_title('Polar trajectory')
23 ax2.tick_params(axis='both', labelsize=6)
24
25 # --- 3. временные зависимости (вся нижняя строка) ---
26 ax3 = fig.add_subplot(gs[1, :])
27 ax3.plot(t, np.sin(t)*np.exp(-0.1*t), '-',
28         t, np.cos(t)*np.exp(-0.1*t), '-.')
29 ax3.set_xlabel('Time, s')
30 ax3.set_ylabel('State space')
31 ax3.legend(['distance, m', 'velocity, m/s'])
32
33 plt.tight_layout()
34 plt.show()

```



Примечание. В “подокне” вообще говоря, можно строить любое изображение, в том числе и трёхмерное. Важно только совместимость типов.

plotly

Библиотека plotly ориентирована на интерактивные графики. В отличие от статичных рисунков, здесь удобно строить наборы связанных графиков в одной фигуре и использовать взаимодействие пользователя (масштабирование, подсказки при наведении, выбор точек и т. п.).

Для размещения нескольких областей рисования используется функция `make_subplots`, которая создаёт один объект `Figure` с несколькими “подокнами”. Продемонстрируем это на примере *зависимого выделения участков*: выбор точек на одном графике подсвечивает соответствующие точки на другом.

```
1 import numpy as np
2 import plotly.graph_objects as go
3 from plotly.subplots import make_subplots
4
5 # данные
6 t = np.arange(0, 4*np.pi, 0.05)
7 distance = np.cos(t) * np.exp(-0.1*t)
8 velocity = np.sin(t) * np.exp(-0.1*t)
9
10 # фигура с двумя подграфиками
11 fig = make_subplots(
12     rows=2, cols=1,
13     shared_xaxes=True,
14     subplot_titles=("Distance vs time", "Velocity vs time")
15 )
16
17 # дистанция
18 fig.add_trace(
19     go.Scatter(x=t, y=distance, mode="lines", name="distance"),
20     row=1, col=1
21 )
22
23 # скорость
24 fig.add_trace(
25     go.Scatter(x=t, y=velocity, mode="lines", name="velocity"),
26     row=2, col=1
27 )
28
29 # выделение интервала [4, 6]
30 fig.add_vrect(
31     x0=4, x1=6,
32     fillcolor="orange",
33     opacity=0.25,
34     line_width=0
35 )
36
37 # оформление осей
38 fig.update_xaxes(title_text="Time, s", row=2, col=1)
```

```

39 fig.update_yaxes(title_text="Distance, m", row=1, col=1)
40 fig.update_yaxes(title_text="Velocity, m/s", row=2, col=1)
41
42 fig.update_layout(
43     height=600,
44     showlegend=False
45 )
46
47 fig.show()

```

В приведённом примере показан минимальный функционал, дающий представление о возможностях библиотеки `plotly`. На официальном сайте <https://plotly.com/> в разделе <https://plotly.com/python/> представлены типовые примеры, реализуемые данной библиотекой.

4.1.5 Диаграммы, гистограммы, вектора и случайные числа

Графики подобного типа могут быть построены с помощью нескольких библиотек.

Примеры построения диаграмм в библиотеке `plotly` приведены на странице <https://plotly.com/python/statistical-charts>.

Библиотека `seaborn` специализирована статистической визуализации и построении информативных графиков. Официальная документация библиотеки доступна по адресу <https://seaborn.pydata.org/>, а примеры использования приведены в разделе <https://seaborn.pydata.org/examples/>.

`matplotlib`

Самая простая в использовании и нетребовательная к ресурсам библиотека — `matplotlib`. Приведём основные функции на примере искусственно сформированных данных с заданным распределением.

Нам понадобятся импорты

```

1 import numpy as np
2 import matplotlib.pyplot as plt

```

см `lesson_3.ipynb`

Глава 5

Численное решение обыкновенных дифференциальных уравнений

[Численное решение ОДУ]

В языке Python основным инструментом для численного решения обыкновенных дифференциальных уравнений (ОДУ) является библиотека SciPy, ее модуль `scipy.integrate`.

Рассмотрим задачу Коши:

$$\frac{dy}{dt} = f(t, y), \quad y(t_0) = y_0.$$

Для её решения используется функция `solve_ivp`.

```
1 from scipy.integrate import solve_ivp
```

5.1 Функция интегрирования

Общий вид вызова:

```
1 solve_ivp(fun, t_span, y0, method=..., t_eval=...)
```

где:

- `fun` — правая часть системы (функция вида $f(t, y)$),
- `t_span` — интервал интегрирования (t_0, t_f),
- `y0` — начальные условия,
- `method` — численный метод,
- `t_eval` — точки, в которых требуется решение.

5.1.1 Численные методы

Функция `solve_ivp` поддерживает следующие методы:

- RK45 — метод Рунге–Кутты 4(5) порядка,

- RK23 — метод Рунге–Кутты 2(3) порядка,
- DOP853 — метод высокого порядка,
- Radau — неявный метод для жёстких систем,
- BDF — метод обратных дифференцирований,
- LSODA — автоматический выбор метода.

5.1.2 Пример

Рассмотрим уравнение:

$$\frac{dy}{dt} = -y, \quad y(0) = 1.$$

где y — вектор длины n .

```
1 from scipy.integrate import solve_ivp
2
3 def f(t, y):
4     return -y
5
6 sol = solve_ivp(f, (0, 5), [1.0])
```

Результат решения содержится в объекте `sol`:

- `sol.t` — значения времени,
- `sol.y` — значения решения,
- `sol.success` — индикатор успешного завершения.

5.1.3 Задание сетки решения

Если требуется получить решение в заданных точках, используется параметр `t_eval`:

```
1 import numpy as np
2
3 t_eval = np.linspace(0, 5, 100)
4
5 sol = solve_ivp(f, (0, 5), [1.0], t_eval=t_eval)
```

5.1.4 События

Функция `solve_ivp` позволяет отслеживать события:

```
1 def event(t, y):
2     return y[0] - 0.5
3
4 event.terminal = True
5 event.direction = -1
6
7 sol = solve_ivp(f, (0, 5), [1.0], events=event)
```

5.2 Краевые задачи

Для решения краевых задач используется функция `solve_bvp`:

```
1 from scipy.integrate import solve_bvp
```

Она предназначена для решения задач вида:

$$\frac{dy}{dx} = f(x, y), \quad g(y(a), y(b)) = 0,$$

где условия задаются на границах отрезка.

5.2.1 Общий вид вызова

```
1 solve_bvp(fun, bc, x, y)
```

где:

- `fun(x, y)` — правая часть системы,
- `bc(ya, yb)` — граничные условия,
- `x` — начальная сетка,
- `y` — начальное приближение решения.

5.2.2 Пример

Рассмотрим краевую задачу:

$$y'' + y = 0, \quad y(0) = 0, \quad y(\pi/2) = 1.$$

Приведём её к системе первого порядка:

$$\begin{cases} y'_1 = y_2, \\ y'_2 = -y_1. \end{cases}$$

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy.integrate import solve_bvp
4
5 # система уравнений
6 def fun(x, y):
7     return np.vstack((y[1], -y[0]))
8
9 # граничные условия
10 def bc(ya, yb):
11     return np.array([ya[0], yb[0] - 1])
12
13 # начальная сетка
14 x = np.linspace(0, np.pi/2, 10)
15
16 # начальное приближение (2 строки: y и y')
```

```

17 y_init = np.zeros((2, x.size))
18
19 # решение
20 sol = solve_bvp(fun, bc, x, y_init)

```

5.2.3 Замечания

- Функция `solve_bvp` использует итерационный метод и требует начального приближения.
- Размерность массива `y` — `(n, len(x))`, где `n` — число уравнений.
- Решение возвращается в виде объекта, содержащего интерполяционную функцию `sol.sol(x)`.
- В отличие от задач Коши, результат чувствителен к начальному приближению.

5.3 Простейший пример численного интегрирования

Перейдём к рассмотрению различных возможностей решения задач численного интегрирования систем обыкновенных дифференциальных уравнений, приведём простой пример, который может послужить образцом для решения 90% задач такого типа.

Будем искать решение уравнения Ван дер Поля:

$$\ddot{x} = -x + 0.3 \cdot \dot{x} \cdot (1 - x^2).$$

при начальных условиях:

$$x(0) = 1; \quad \dot{x}(0) = 0.$$

Сделаем замену переменных $y_1 = x$ и $y_2 = \dot{x}$. Преобразуем задачу к виду:

$$\begin{cases} y_1' = y_2, \\ y_2' = -y_1 + 0.3 \cdot y_2 \cdot (1 - y_1^2), \\ y_1(0) = 1, \\ y_2(0) = 0. \end{cases}$$

Для этой системы создадим функцию вычисления правых частей с описанием типа `f(t, y)`:

```

1 # правая часть системы
2 def vanderp(t, y):
3     y1, y2 = y
4     dy1 = y2
5     dy2 = -y1 + 0.3 * y2 * (1 - y1**2)
6     return [dy1, dy2]

```

Далее для численного решения используем функцию `solve_ivp`:

```

1 # начальные условия
2 y0 = [1, 0]
3
4 # интервал интегрирования
5 t_span = (0, 30)
6
7 # точки для вывода решения
8 t_eval = np.linspace(0, 30, 1000)
9
10 # решение
11 sol = solve_ivp(vanderp, t_span, y0, t_eval=t_eval)

```

В результате получим решение на заданном интервале времени:

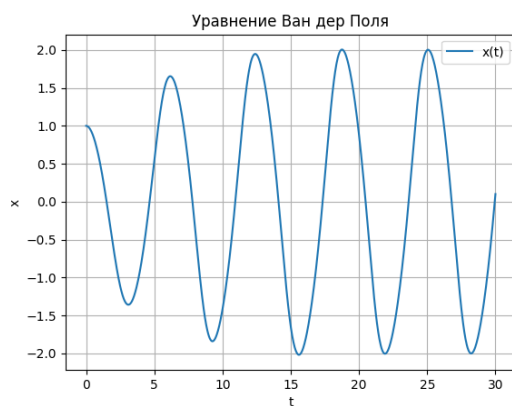


Рис. 5.1: *
Решение $x(t)$

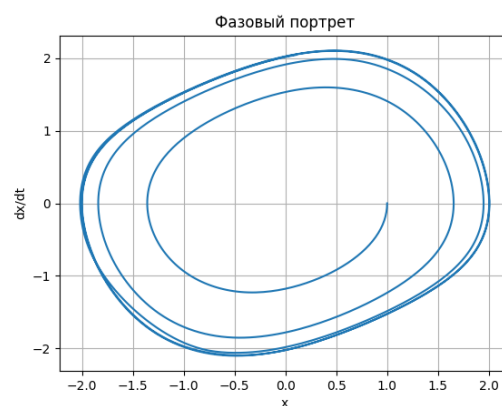


Рис. 5.2: *
Фазовый портрет

5.3.1 Опции решателя

Функция `solve_ivp` решает задачу Коши для системы обыкновенных дифференциальных уравнений и предоставляет широкий набор параметров, позволяющих управлять точностью вычислений, форматом вывода решения и поведением алгоритма.

Общий вид вызова функции:

```

solve_ivp(fun, t_span, y0, method='RK45', t_eval=None,
          dense_output=False, events=None, vectorized=False,
          args=None, **options)

```

Функция `solve_ivp` численно интегрирует систему ОДУ с начальными условиями:

$$\frac{dy}{dt} = f(t, y), \quad y(t_0) = y_0.$$

Здесь t — скалярная независимая переменная (время), $y(t)$ — векторная функция состояния, $f(t, y)$ — векторная функция, задающая систему уравнений.

Некоторые методы поддерживают комплексную область. Для жёстких систем правая часть должна быть комплексно-дифференцируема (удовлетворять условиям

Коши–Римана). Для работы в комплексной области необходимо задать y_0 комплексного типа или переписать задачу в вещественных переменных.

Основные параметры решателя:

- `fun(t, y)` — правая часть системы дифференциальных уравнений. Должна возвращать массив той же размерности, что и y . При использовании `args` принимает дополнительные параметры.
- `t_span = (t_0, t_f)` — интервал интегрирования.
- `y0` — начальное состояние системы.
- `method` — численный метод интегрирования:
 - RK45 — явный метод Рунге–Кутты 5(4),
 - RK23 — метод Рунге–Кутты 3(2),
 - DOP853 — метод 8-го порядка,
 - Radau — неявный метод Рунге–Кутты,
 - BDF — многошаговый метод,
 - LSODA — автоматический выбор метода.

Явные методы применяются для нежёстких задач, неявные — для жёстких.

- `rtol` — относительная погрешность (по умолчанию 10^{-3}).
- `atol` — абсолютная погрешность (по умолчанию 10^{-6}). Может задаваться как скаляр или вектор.

Оценка локальной ошибки имеет вид:

$$|e_i| \leq \text{atol}_i + \text{rtol} \cdot |y_i|.$$

- `t_eval` — массив точек, в которых требуется сохранить решение.
- `dense_output` — логический параметр. При `True` возвращается непрерывное решение в виде функции `sol.sol(t)`.
- `events` — функции событий. Каждое событие определяется как нуль функции

$$\text{event}(t, y) = 0.$$

Дополнительные атрибуты:

- `terminal` — остановка интегрирования при событии,
- `direction` — направление пересечения нуля.
- `vectorized` — указывает, может ли функция `fun` работать с массивами состояний.
- `args` — дополнительные аргументы, передаваемые в `fun` и функции событий.
- Дополнительные параметры (`**options`):

- `first_step` — начальный шаг интегрирования,
- `max_step` — максимальный шаг,
- `min_step` — минимальный шаг (для LSODA),
- `jac` — матрица или функция Якоби,
- `jac_sparsity` — структура разреженности Якобиана,
- `lband, uband` — параметры ленточной структуры.

Результат работы функции возвращается в виде объекта со следующими основными полями:

- `t` — точки времени,
- `y` — значения решения,
- `sol` — непрерывное решение или `None`,
- `t_events, y_events` — информация о событиях,
- `nfev, njev, nlu` — статистика вычислений,
- `status` — код завершения ($-1, 0, 1$),
 - -1 — ошибка интегрирования,
 - 0 — достигнут конец интервала,
 - 1 — остановка по событию.
- `message` — текстовое описание,
- `success` — логический индикатор успешности.

5.4 Пример моделирования механической системы с сухим трением

В качестве примера рассмотрим задачу о движении тела под действием силы сухого трения. Будем считать, что тело массы m , установленное на транспортере, движется с постоянной скоростью v . Тело соединено с неподвижной опорой пружиной с коэффициентом жёсткости k . Между телом и транспортером действует сила сухого трения. Положение тела опишем координатой x .

Введем положительное направление движения груза, сонаправленное вектору скорости ленты, т.е. скорость ленты всегда положительна.

Силу сухого трения будем описывать соотношениями:

$$F = \begin{cases} -\mu N \cdot \text{sign}(\dot{x} - v), & \dot{x} - v \neq 0, \\ R, & \dot{x} - v = 0, \quad |R| \leq \mu N, \end{cases}$$

где μ — коэффициент сухого трения, $N = mg$ — нормальная реакция опоры, R — сила реакции, обеспечивающая прилипание.

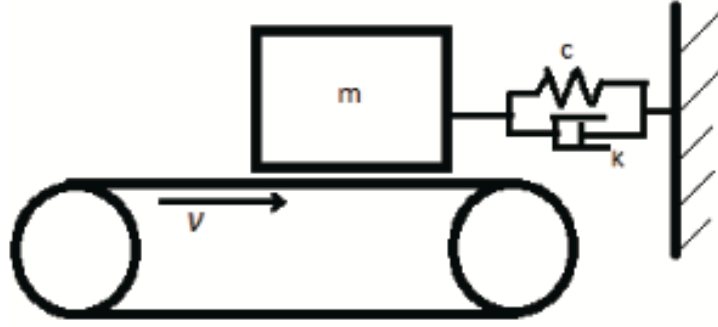


Рис. 5.3: Моделируемая механическая система

Моделирование движения тела сводится к интегрированию уравнения:

$$\ddot{x} = -\frac{k}{m}x - \mu g \cdot \text{sign}(\dot{x} - v),$$

пока выполняется условие скольжения:

$$\dot{x} - v \neq 0,$$

или при $\dot{x} - v = 0$, если $|kx| > \mu N$.

Сделаем замену переменных:

$$y_1 = x, \quad y_2 = \dot{x}.$$

Тогда система примет вид:

$$\begin{cases} y_1' = y_2, \\ y_2' = -\frac{k}{m}y_1 - \mu g \cdot \text{sign}(y_2 - v). \end{cases}$$

при выполнении условий

$$\begin{cases} y_2 - v \neq 0, \\ y_2 - v = 0, \quad \text{при} \quad |ky_1| > \mu N. \end{cases}$$

5.4.1 Общий алгоритм решения задачи

У нашего тела может быть два режима движения: по ленте и вместе с лентой. В общем случае мы могли бы интегрировать систему при любых условиях. Однако в рассматриваемой задаче при движении вместе с лентой система имеет аналитическое решение. Его использование упрощает решение и уменьшает время численного моделирования.

Для формулировки условия прилипания тела к ленте введем

$$q = cx + k\dot{x}.$$

Тогда при $|q| < \mu mg$ тело движется вместе с лентой, а скольжение происходит при $|q| \geq \mu mg$.

При движении груза вместе с лентой его движение описывается следующими соотношениями:

$$x_m = \frac{\mu N - kv}{c}. \quad (5.1)$$

Время движения тела вместе с лентой транспортера:

$$t_p = \frac{x_m}{v} = \frac{\mu N - kv}{cv}. \quad (5.2)$$

5.4.2 Вспомогательные методы контроля

Реализуем решение. Важный момент: для иллюстрации можно выводить графики на каждом логическом шаге алгоритма, однако следует помнить, что графические окна не всегда удобны, т.к. обращение к ним требует сравнительно большого времени. Во многих случаях лучше сначала выполнить расчёты, сохранить результаты, а затем отдельно построить иллюстрации.

В задачах, где вычисления длительны или объём результатов слишком велик для их хранения в оперативной памяти, полезно сохранять промежуточные данные на диск. Для массивов NumPy можно использовать функции `np.save`, `np.savez` и `np.savez_compressed`; загрузка выполняется функцией `np.load`. Формат `.npy` предназначен для одного массива, а `.npz` — для набора массивов.

Если результаты имеют табличную структуру, можно использовать формат `Parquet`, например, через пакет `pandas`. Для сохранения и загрузки применяются функции `to_parquet` и `read_parquet`.

Для сериализации произвольных объектов Python можно использовать стандартный модуль `pickle`. Он удобен для сохранения словарей, списков и более сложных структур данных, но требует осторожности при загрузке файлов.

Для отладки всегда можно использовать печать сообщений и промежуточных результатов. Однако в более сложных проектах предпочтительнее использовать механизм логирования.

Логирование позволяет:

- разделять сообщения по уровням важности (отладка, информация, предупреждения, ошибки);
- гибко управлять выводом (в консоль, файл или оба варианта);
- сохранять историю работы программы.

В Python для этого используется стандартный модуль `logging`.

```
1 import logging
2
3 # настройка логгера
4 logging.basicConfig(
5     level=logging.INFO, # уровень логирования
6     format="%(asctime)s [%(levelname)s] %(message)s",
7     handlers=[
8         logging.FileHandler("app.log"), # запись в файл
9         logging.StreamHandler()        # вывод в консоль
10 ]
11 )
```

```

12
13 logger = logging.getLogger(__name__)
14
15 # примеры сообщений
16 logger.debug("отладочное сообщение")
17 logger.info("программа запущена")
18 logger.warning("предупреждение")
19 logger.error("ошибка")
20 logger.critical("критическая ошибка")

```

Сообщения уровня `DEBUG` используются для детальной отладки, `INFO` — для общей информации о ходе программы, `WARNING` — для предупреждений, `ERROR` и `CRITICAL` — для ошибок.

При указанной настройке все сообщения уровня `INFO` и выше будут одновременно выводиться в консоль и сохраняться в файл `app.log`.

5.4.3 Реализация решения

Для реализации решения нам потребуются пакеты:

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy.integrate import solve_ivp

```

Параметры системы:

```

1 # неизменяемые параметры системы
2 c = 1.0      # коэффициент упругости
3 k = 0.001    # коэффициент демпфирования
4 m = 1.0      # масса
5 g = 9.8      # ускорение свободного падения
6 mu = 0.1     # коэффициент трения
7 v = 4.0      # скорость ленты, для этого примера величина строго больше 0
8
9 # параметры в кортеже
10 sys_args = (c, k, m, g, mu, v)

```

Обратите внимание на строку с кортежем. Кортеж — это неизменяемая структура данных, которая позволяет компактно передавать параметры в функции.

Теоретически в среде блокнота (например, Jupyter) переменные, объявленные на верхнем уровне, являются глобальными, то есть доступны во всех ячейках. Также можно явно использовать глобальные переменные среды с помощью ключевого слова `global`.

Рассмотрим пример функции, которая будет возвращать один и тот же результат в обоих вариантах — удвоенную передаваемую величину x .

В этом примере в функции `change` указано, что `c` надо взять из внешнего окружения.

```
1 c = 1.0
2 def change(x):
3     global c
4     c = 2.0
5     return x*c
```

`c` изменится для всего проекта.

При использовании `global` возникают следующие проблемы:

- функции становятся зависимыми от внешнего состояния;
- усложняется повторное использование кода;
- повышается вероятность ошибок при изменении переменных;
- затрудняется отладка и тестирование.

Использование передачи параметров представляется предпочтительным, так как не обладает вышеприведёнными недостатками и считается хорошим тоном.

Задаём варьируемые параметры:

```
1 # интервал времени
2 T0 = 0.0
3 Tf = 30.0
4 # начальные условия [координата, скорость]
5 y0 = np.array([0, 6.0], dtype=float)
```

Эти параметры предлагается варьировать самостоятельно.

Основные функции, которые нужны для численного интегрирования: функция правой части `ft` и функция, задающая события `events`.

```
1 # правая часть на участке скольжения
2 def ft(t, y, c, k, m, g, mu, v, sgn):
3     f = np.zeros(2, dtype=float)
4     f[0] = y[1]
5     f[1] = -c / m * y[0] - k / m * y[1] - mu * g * sgn
6     return f
7
8 # событие: скорость тела сравнялась со скоростью ленты
9 def events(t, y, c, k, m, g, mu, v, sgn):
10    return y[1] - v
```

Теперь основное тело кода.

```
1 # настройка срабатывания событий
2 events.terminal = True # по событию идет остановка интегрирования
3 events.direction = 0 # смена знака в любом направлении
4 T_parts = [] # накопители решения
5 Y_parts = []
6 t_cur = T0 # начальные условия
7 y_cur = y0.copy()
8 eps_t = 1e-9
```

Если не указывать, что `c` надо взять из внешнего окружения, то в этом примере `c` — локальная переменная.

```
1 c = 1.0
2
3 def change(x):
4     c = 2.0
5     return x*c
```

`c` останется равным 1.

Обратите внимание, что для задания начальных условий интегрирования используется копирование `y0`. В Python переменные являются ссылками на объекты, поэтому при присваивании `y_cur = y0` обе переменные будут указывать на один и тот же объект. В результате изменение `y_cur` приведёт к изменению `y0`.

Далее построим цикл по времени. Обратите внимание, что подобные циклы при наличии ошибок могут становиться бесконечными. На этапе отладки можно ввести дополнительную переменную — счётчик шагов — и использовать прерывание `break` при превышении некоторого заданного значения. При отладке больших проектов рекомендуется использовать логирование.

```

1 while t_cur < Tf:
2     # текущий знак силы трения на участке скольжения
3     sgn = np.sign(y_cur[1] - v)
4     if sgn == 0:
5         f0 = ft(t_cur, y_cur, c, k, m, g, mu, v, sgn)
6         # обновляем начальные условия
7         y_cur = y_cur + dt * np.array(f0)
8         t_cur = t_cur + dt
9         sgn = np.sign(y_cur[1] - v)

```

Остановимся здесь на двух моментах. Первый состоит в том, что функция `np.sign` возвращает одно из трёх значений:

$$\text{sign}(x) = \begin{cases} -1, & x < 0, \\ 0, & x = 0, \\ 1, & x > 0. \end{cases}$$

Попав в подобную точку, выполним один шаг интегрирования и обновим начальные условия. Такой приём потребуется и далее, поскольку функция `solve_ivp` допускает срабатывание `events` уже на первом шаге. Таким образом, если интегрирование начинается на границе, необходимо выполнить один шаг, чтобы сдвинуть начальные условия с этой границы. В противном случае срабатывание прерывания интегрирования на первом же шаге приведёт к заикливанию.

```

1     # интегрируем участок скольжения до события y2 = v или до конца интервала
2     sol_slip = solve_ivp(
3         ft,
4         (t_cur, Tf),
5         y_cur,
6         method="RK45",
7         events=events,
8         args=(sys_args, sgn),
9         dense_output=False,
10        max_step=dt,
11    )
12    T_parts.append(sol_slip.t)
13    Y_parts.append(sol_slip.y)

```

Метод изменения списка `.append` имеет амортизированную константную сложность, однако при работе с большими объёмами данных или при большом числе операций может приводить к заметным накладным расходам. В таких случаях рекомендуется использовать более эффективные структуры данных или заранее выделять память.

Если прерывание интегрирования произошло на событии, т.е. сравнялись скорости ленты и груза, то в возвращаемой структуре решения будет время и состояние системы в момент события. Для наглядности выделим их в отдельные переменные:

```
1 t_hit = sol_slip.t_events[0][0]
2 y_hit = sol_slip.y_events[0][0].copy()
```

Далее требуется проверить условие прилипания и либо продолжить интегрирование (отступив от состояния срабатывания `events`), либо смоделировать движение груза вместе с лентой.

Для условия прилипания введём переменную `q_stick`. Это не обязательно, так как вычисление выполняется один раз, и в данном примере сделано скорее для наглядности и читаемости кода. Здесь учитывается, что скорость груза равна скорости ленты.

```
1 q_stick = c * y_hit[0] + k * v
```

Реализуем случаи скольжения/прилипания с помощью операторов `if/else`. Если условия прилипания не выполнены, то обновляем начальные условия и смещаем их.

```
1 if np.abs(q_stick) > mu * m * g:
2     t_cur = t_hit
3     y_cur = y_hit.copy()
4     f0 = ft(t_cur, y_cur, c, k, m, g, mu, v, sgn)
5     y_cur = y_cur + dt * np.array(f0)
6     t_cur = t_cur + dt
```

Случай, когда выполнены условия прилипания. Тогда крайнее положение груза задаётся соотношением (5.1), а продолжительность движения груза вместе с лентой — (5.2).

```
1 else: # при прилипании
2     # момент выхода из прилипания и координата определяется аналитически
3     if np.abs(v) < 1e-15: # частный случай: лента неподвижна
4         t_leave = Tf
5     else:
6         x_m = (mu * m * g - k * v) / c
7         tau = (x_m - y_hit[0]) / v
8         t_leave = t_hit + tau
9         # строим аналитический участок
10        T_parts.append(t_leave)
11        Y_stick = np.vstack((x_m, v))
12        Y_parts.append(Y_stick)
13        # условия для нового скольжения
14        y_cur = np.array([x_m, v], dtype=float)
15        t_cur = t_leave
```

Результат интегрирования данной системы:

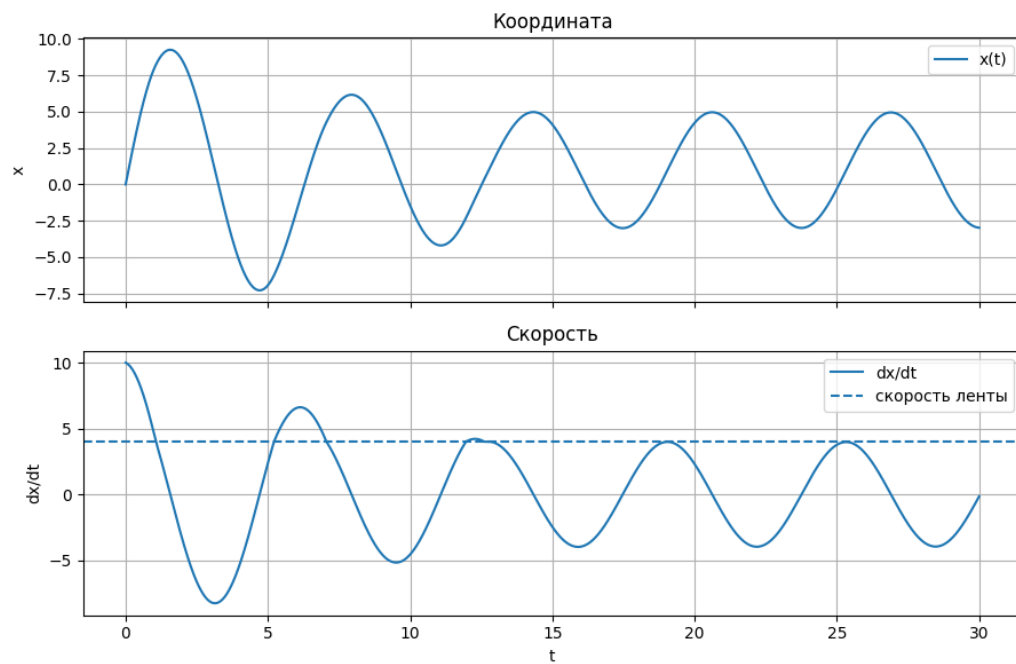


Рис. 5.4: Траектории

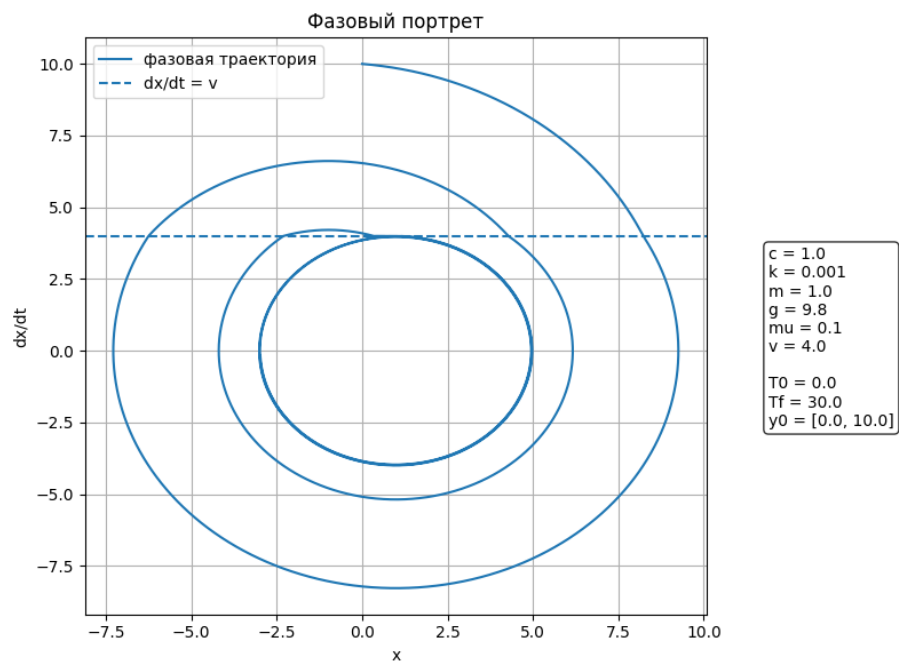


Рис. 5.5: Фазовый портрет